

1. Explain lazy evaluation in PySpark.

Lazy evaluation means Spark does not execute transformations immediately. It builds a logical execution plan and executes it only when an **action** like `show()`, `count()`, or `write()` is called. This helps Spark optimize the execution plan.

2. How does caching work in PySpark?

Caching stores a DataFrame/RDD in memory so it can be reused without recomputing it.
Example:

```
df.cache()
```

```
df.count()
```

For memory and disk:

```
df.persist()
```

3. Difference between wide and narrow transformations?

- **Narrow:** Data does not need to move between partitions. Example: `filter()`, `map()`.
- **Wide:** Data moves between partitions and causes a **shuffle**. Example: `groupBy()`, `join()`, `orderBy()`.

4. How do you optimize query performance in Azure SQL Database?

I use:

- Proper indexes
- Query optimization
- Avoid `SELECT *`
- Check execution plans
- Update statistics
- Avoid unnecessary joins
- Use appropriate partitioning where applicable
- Monitor long-running queries

5. Describe integrating ADF with Azure Synapse Analytics.

ADF can extract data from different sources and load it into Synapse. I create **Linked Services** → **Datasets** → **Pipeline** → **Copy Activity**, and configure triggers, parameters, monitoring, and error handling.

6. How do you handle schema evolution in Azure Data Lake?

I use formats like **Delta Lake** that support schema evolution. In Databricks, I can use options such as:

```
.option("mergeSchema", "true")
```

I also validate incoming schemas and handle incompatible changes separately.

7. SQL query to find the nth highest salary.

Using DENSE_RANK():

```
SELECT salary
```

```
FROM (
```

```
    SELECT salary,
```

```
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
```

```
    FROM employee
```

```
) t
```

```
WHERE rnk = 3;
```

Here, 3 means third-highest salary.

8. How do you implement CI/CD for ADF and Databricks?

I use **Azure DevOps** with Git for version control. Code is developed in a feature branch, reviewed and merged, then CI/CD pipelines deploy ADF and Databricks changes across **Dev** → **Test** → **Production** using parameterized configurations.

9. PySpark code to calculate total sales for each product category.

```
from pyspark.sql.functions import sum
```

```
result = df.groupBy("product_category") \
```

```
    .agg(sum("sales").alias("total_sales"))
```

```
result.show()
```

10. How do broadcast joins improve PySpark performance?

A broadcast join sends the smaller DataFrame to all executors. This avoids a large shuffle and makes joins faster when one dataset is small.

```
from pyspark.sql.functions import broadcast
```

```
result = large_df.join(  
    broadcast(small_df),  
    "id"  
)
```

11. Role of driver and executor in Spark architecture.

- **Driver:** Runs the main application, creates the SparkContext, builds the execution plan, and coordinates jobs.
- **Executors:** Run the actual tasks on worker nodes and store intermediate/cache data.

12. How do you manage and monitor ADF pipeline performance?

I use **ADF Monitor** to check pipeline runs, activity duration, failures, and throughput. I optimize activities, parallelize independent tasks, tune Integration Runtime, and configure alerts for failures.

13. SQL query to find employees earning more than department average.

```
SELECT *  
  
FROM employee e  
  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employee  
    WHERE department_id = e.department_id  
);
```

14. Explain Delta Lake and its advantages.

Delta Lake is a storage layer built on data lake storage that provides **ACID transactions** and reliable data management.

Advantages:

- ACID transactions
- Schema enforcement
- Schema evolution

- Time travel
- MERGE/UPSERT
- Better reliability

15. How do you implement schema drift handling in ADF?

I use **Mapping Data Flows** with schema drift enabled. This allows ADF to handle columns that are added or removed dynamically without manually mapping every column.

16. Python code to check whether a number is a palindrome.

```
num = 121
```

```
if str(num) == str(num)[::-1]:
```

```
    print("Palindrome")
```

```
else:
```

```
    print("Not Palindrome")
```

17. Significance of Z-Ordering in Delta tables.

Z-Ordering colocates related data in storage based on selected columns. It improves **data skipping** and reduces the amount of data that Databricks needs to scan.

Example:

```
OPTIMIZE sales
```

```
ZORDER BY (customer_id);
```

18. How do you handle incremental data load in Databricks?

I use a **watermark/incremental column** such as last_updated_timestamp or use **Auto Loader** for files arriving in ADLS. I process only new or changed data and maintain checkpoints/control tables.

19. Explain Adaptive Query Execution (AQE) in Spark.

AQE dynamically changes the query execution plan at runtime based on actual data statistics. It can handle issues like **data skew, shuffle partition optimization, and join strategy optimization**.

20. How do you optimize data partitioning in ADLS?

I choose partition columns based on common query filters, such as date. I avoid too many small partitions and use a balanced partition strategy.

Example:

/year=2026/month=09/day=05/

21. Process of creating a real-time analytics pipeline.

Typical architecture:

Source → Event Hubs/Kafka → Stream Processing → Delta Lake → Synapse/Databricks → Power BI

I configure streaming ingestion, transformations, checkpoints, error handling, and monitoring.

22. PySpark code for a left join between two DataFrames.

```
result = df1.join(  
    df2,  
    df1.id == df2.id,  
    "left"  
)
```

```
result.show()
```

23. Security best practices for Azure Data Lake.

I use:

- Azure AD authentication
- RBAC
- ACLs
- Managed identities
- Azure Key Vault for secrets
- Private endpoints
- Encryption
- Least-privilege access

24. Explain Integration Runtime (IR) in ADF.

Integration Runtime is the **compute infrastructure used by ADF** to perform data movement and transformation activities.

Main types:

- **Azure IR**
- **Self-hosted IR**
- **Azure-SSIS IR**

25. How do you design a fault-tolerant architecture for big data processing?

I use:

- Checkpointing
- Retry mechanisms
- Idempotent processing
- Error handling
- Dead-letter/error storage
- Monitoring and alerts
- Replication/redundancy
- Delta Lake ACID transactions